

GradHPO: Making Gradient-Based Hyperparameter Optimization Practical in JAX

GradHPO: Making Gradient-Based Hyperparameter Optimization Practical in JAX

Hyperparameter tuning is one of the most expensive and least elegant parts of machine learning. We choose a learning rate, a regularization coefficient, a data-weighting rule, maybe even an architecture parameter. Then we train a model, evaluate it, change the setting, and repeat.

Classical tools treat this as a black-box search problem. Grid search is simple but quickly explodes. Random search is more flexible but still blind. Bayesian optimization is smarter, but it usually sees training as a function to query, not as a differentiable process.

GradHPO takes a different route. It is a Python library for **gradient-based hyperparameter optimization** in **JAX**. Instead of asking only “which setting worked best in past trials?”, it asks: **how would the validation loss change if we slightly changed the hyperparameters?**

That question leads to hypergradients, bilevel optimization, and several practical approximations. GradHPO packages these ideas behind a common interface and makes them easier to try in real experiments.

The core idea: training has two levels

Hyperparameter optimization can be viewed as a two-level problem.

At the **inner level**, we train model parameters w on the training data. At the **outer level**, we update hyperparameters λ to improve validation performance. A compact picture is:

$$\text{train model: } w_T = \Phi_T(w_0, \lambda), \quad \text{choose hyperparameters: } \min_{\lambda} L_{val}(w_T, \lambda).$$

The hard part is that w_T depends on λ . Changing a learning rate, regularization weight, or data weight changes the whole training trajectory, not only the final validation formula.

So the validation loss depends on hyperparameters in two ways: directly through λ , and indirectly through the trained model w_T . Exact hypergradients try to account for both effects, but doing this exactly may require differentiating through many training steps and computing expensive second-order information. For large neural networks, that is often too slow or too memory-hungry.

GradHPO focuses on the practical middle ground: keep useful hypergradient information, but avoid the full cost of exact unrolling.

What GradHPO includes

GradHPO collects several gradient-based HPO algorithms under a single JAX-oriented interface.

Method	Intuition	Best use case
HyperDistill	Compresses long-horizon hypergradient information into a cheaper online estimate.	Online updates and meta-learning-like settings.
T1-T2 + DARTS	Uses a one-step approximation with a finite-difference estimate of the second-order term.	Fast experiments and cheap second-order baselines.
Generalized Greedy	Accumulates discounted local hypergradient information across the inner trajectory.	Problems where the full training path matters.
FO baseline	Uses only the direct first-order gradient.	Sanity checks and inexpensive baselines.
One-Step baseline	Performs a simple one-step lookahead.	Comparing short-horizon approximations.

This design is useful because the methods are interchangeable. A user can start with a cheap baseline, then switch to a richer approximation without rewriting the whole training pipeline.

Three algorithmic ideas, without too much math

HyperDistill is designed for online hyperparameter updates. It keeps a compact moving-average summary of the training trajectory, sometimes called a distilled weight point. This gives the hypergradient some memory of past training steps without storing every intermediate model state.

T1-T2 + DARTS is the lightweight option. It asks how the hyperparameters affect the next update, rather than the entire future trajectory. GradHPO estimates the needed second-order information numerically, in the spirit of DARTS: perturb, observe the difference, and use that as a cheap derivative estimate.

Generalized Greedy sits between these extremes. It accumulates local hypergradient contributions from several inner steps and discounts older ones with a parameter such as γ . Small γ behaves more like a one-step method; larger γ listens more to the earlier training path.

The trade-off is clear: richer trajectory information can improve the hypergradient, but it costs more computation.

Why JAX is a natural fit

GradHPO is built around **JAX**, and that is more than an implementation detail. Bilevel optimization needs differentiable update functions, vector-Jacobian products, compilation, vectorization, and clean handling of nested parameter structures. JAX provides these through `grad`, `vjp`, `jit`, `vmap`, and `pytrees`.

The library also uses **Optax**, which makes optimizer definitions composable. This is important because bilevel optimization usually involves two optimizers at once: one for model parameters and another for hyperparameters.

Pytrees are especially helpful. A hyperparameter does not need to be a single scalar. It can be a nested object with the same structure as the model weights. For example, a per-parameter learning-rate vector can mirror the structure of the neural network itself, avoiding manual flattening and indexing.

PyPI: why the package page matters

GradHPO is available on PyPI, so installation is simple:

```
pip install gradhpo
```

At the time of writing, the PyPI page lists version **0.1.2**, released on **April 30, 2026**, with Python **>= 3.9** support. It also links to the source repository, documentation, and issue tracker.

This creates several practical opportunities:

- **Easy adoption:** users can install the library without cloning the repository.
- **Reproducibility:** experiments can pin a version, for example `gradhpo==0.1.2`.
- **Teaching:** students can install the same package and focus on the ideas.
- **Research comparison:** new methods can be compared against packaged baselines.
- **Open-source workflow:** metadata points to docs, source code, license, and issues.

The project is marked as an alpha-stage research package, so it is best understood as an experimental toolkit rather than a mature production framework. For this topic, that is reasonable: hypergradient methods are still an active research area.

A minimal usage pattern looks like this:

```
import jax.numpy as jnp
from gradhpo import OnlineHypergradientOptimizer

params = {"w": jnp.zeros(10)}
hyperparams = {"log_lam": jnp.array(0.0)}

optimizer = OnlineHypergradientOptimizer(
    update_fn=update_fn,
    gamma=0.99,
    estimation_period=10,
    T=20,
)

state = optimizer.init(params, hyperparams)
```

The exact code depends on the task, but the pattern is stable: define a differentiable training update, define train and validation losses, choose a bilevel optimizer, and update hyperparameters using validation feedback.

What the demo shows

The attached demo notebook is valuable because it turns the theory into concrete tasks.

Task 1: data hyper-cleaning. The demo creates a noisy training set and assigns a learnable weight to each training example. The inner loop trains a small MLP on the weighted loss. The outer loop uses a clean validation set to decide which samples should be trusted. The goal is interpretable: clean examples should receive higher weights, while corrupted examples should be down-weighted.

Task 2: per-parameter learning-rate meta-learning. Each scalar model parameter receives its own learnable learning rate. This is a small version of learning the optimizer itself. It also demonstrates why pytrees are useful: hyperparameters can follow the same nested structure as model parameters.

Task 3: joint L1 and L2 regularization tuning. The notebook tunes two regularization coefficients and compares the optimization trajectory with a grid-search oracle. Because the hyperparameter space is two-dimensional, the path can be visualized on a contour plot, which makes the behavior easy to explain.

Benchmark section. The final section compares algorithms by quality and cost. This is the right framing: there is no universally best hypergradient approximation. Cheap methods are good for prototyping; trajectory-aware methods are better when hyperparameter quality matters more than runtime.

Task 1: Data Hyper-Cleaning (30% label noise)

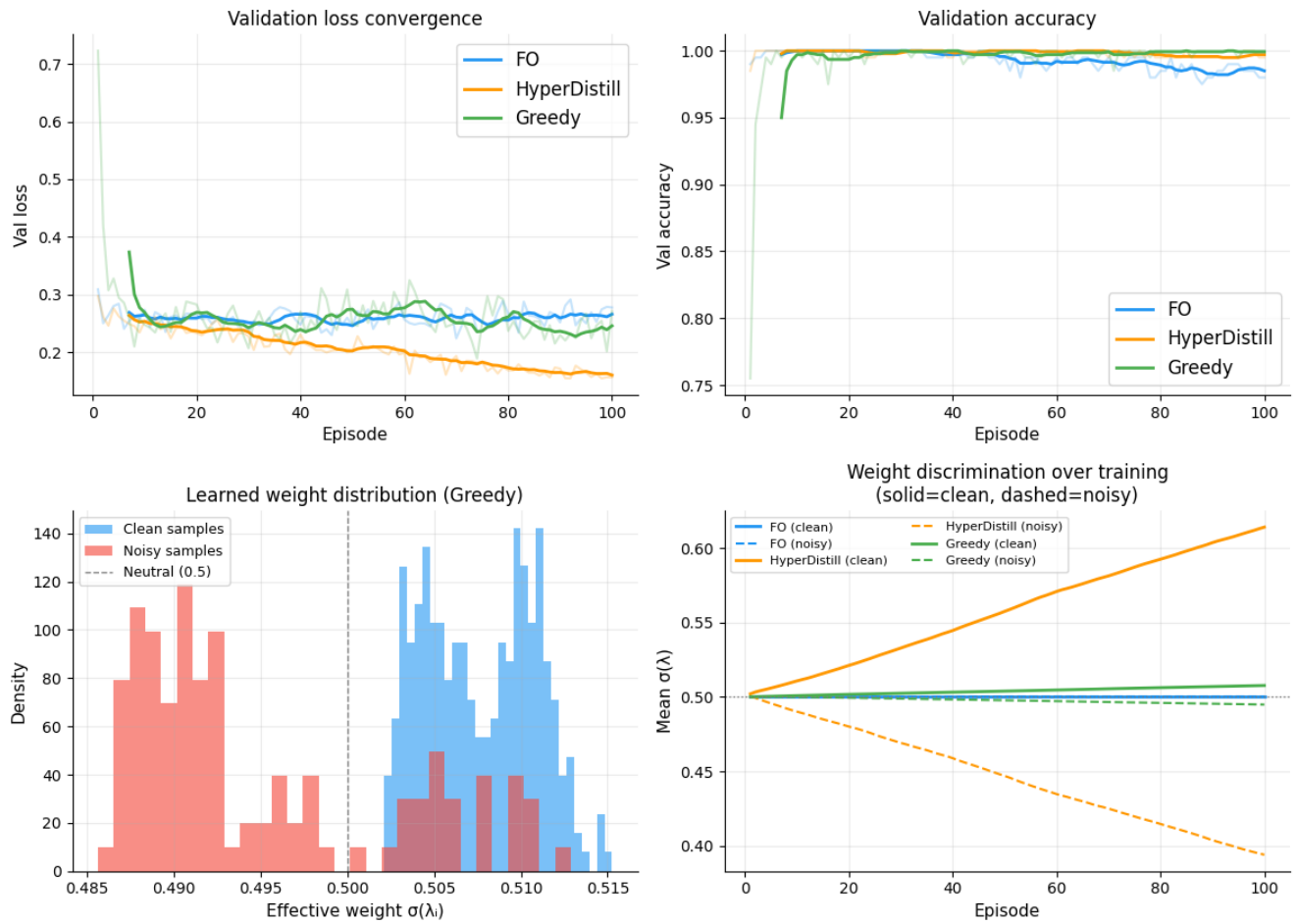


Figure 1: A demo run for data hyper-cleaning. The figure compares validation loss, validation accuracy, learned weight distribution, and the separation between clean and noisy sample weights.

When should you use GradHPO?

GradHPO is most relevant when hyperparameters are differentiable and numerous. Examples include per-sample data weights, per-layer or per-parameter learning rates, differentiable regularization strengths, meta-learning settings, and continuous relaxations of architecture or training-rule parameters.

It is less suitable when hyperparameters are purely discrete, when training is not differentiable, or when a simple manually chosen setting already works well.

A practical starting path is:

1. Install the package from PyPI.
2. Run the demo notebook and inspect the plots.
3. Start with `F0Optimizer` or `OneStepOptimizer` as sanity checks.
4. Move to `T1T2Optimizer` for a cheap second-order approximation.
5. Try `OnlineHypergradientOptimizer` when online updates matter.
6. Try `GreedyOptimizer` when long-range trajectory effects are important.

Final thoughts

GradHPO is interesting because it sits between theory and usable software. The theory says that hyperparameters can be optimized with gradients if we account for training dynamics. Engineering reality says exact hypergradients are often too expensive.

The library's strength is not one magic optimizer. Its strength is that it makes several meaningful approximations accessible through one JAX-native interface. For researchers, this makes comparison easier. For students, it makes bilevel optimization less abstract. For JAX users, it provides a compact toolkit for moving beyond black-box hyperparameter search.

Useful links

- GradHPO on GitHub: <https://github.com/intsystems/gradhpo>
- GradHPO on PyPI: <https://pypi.org/project/gradhpo/>
- Documentation: <https://intsystems.github.io/gradhpo/>
- JAX documentation: <https://jax.readthedocs.io/>
- Optax documentation: <https://optax.readthedocs.io/>